

BÀI THỰC HÀNH SỐ 5

I. Mục tiêu

Sau khi hoàn thành bài Lab, sinh viên có thể:

1. Kiến thức

- Nguyên lý gom cụm: Hiểu cơ chế hoạt động của các thuật toán: K-Means, DBSCAN, Gaussian Mixture Model (GMM) và phân cấp (Agglomerative Clustering).
- Đánh giá mô hình: Nắm vững các tiêu chí đo lường hiệu quả phân cụm như Silhouette Score, Elbow Method, BIC và so sánh đối chiếu với nhãn thật (Ground Truth).

2. Kỹ năng

- Xử lý dữ liệu: Thành thạo tiền xử lý, chuẩn hóa (StandardScaler) và giảm chiều dữ liệu (PCA) trong môi trường Python.
- Thực thi thuật toán: Xây dựng, huấn luyện và tinh chỉnh tham số (Grid Search) cho các mô hình gom cụm.
- Trực quan hóa và phân tích: Trực quan hóa dữ liệu đa chiều, diễn giải các biểu đồ chuyên sâu (k-distance graph, dendrogram) và đánh giá, so sánh hiệu quả giữa các thuật toán trong bài toán thực tế.

II. Bài toán gom cụm

1. Gom cụm là gì?

Gom cụm (clustering) là một thuật toán **học máy không giám sát**, có nhiệm vụ tổ chức và phân loại các đối tượng, điểm dữ liệu hoặc quan sát khác nhau thành các nhóm hoặc cụm **dựa trên sự tương đồng** hoặc các mẫu.



- Cluster 1: High income/high property value
- Cluster 2: Middle income/middle property value
- Cluster 3: High income/low property value

Ứng dụng trong phân khúc thị trường, gom cụm giúp chia khách hàng thành các nhóm có đặc điểm tương đồng dựa trên dữ liệu như thu nhập, tài sản,...

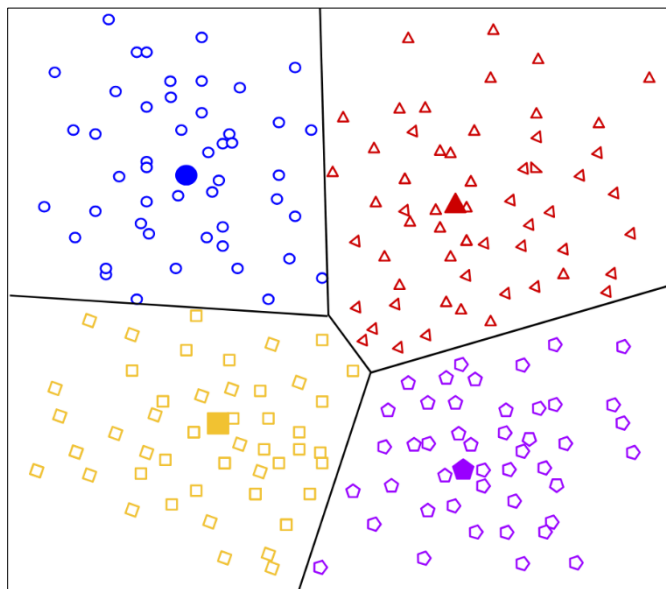
Ví dụ có thể hình thành các nhóm như:

- Thu nhập cao, có nhà
- Trung lưu, có nhà
- Thu nhập cao, chưa có nhà

Quá trình gom cụm có thể sử dụng chỉ một đặc trưng của dữ liệu hoặc sử dụng tất cả các đặc trưng có trong dữ liệu

2. Phân loại các bài toán gom cụm

a. Gom cụm dựa trên tâm (Centroid-based)



Đây là phương pháp phân chia dữ liệu thành các nhóm dựa trên khoảng cách giữa các điểm dữ liệu và tâm cụm.

Tâm cụm có thể là giá trị trung bình hoặc trung vị của các điểm trong cụm.

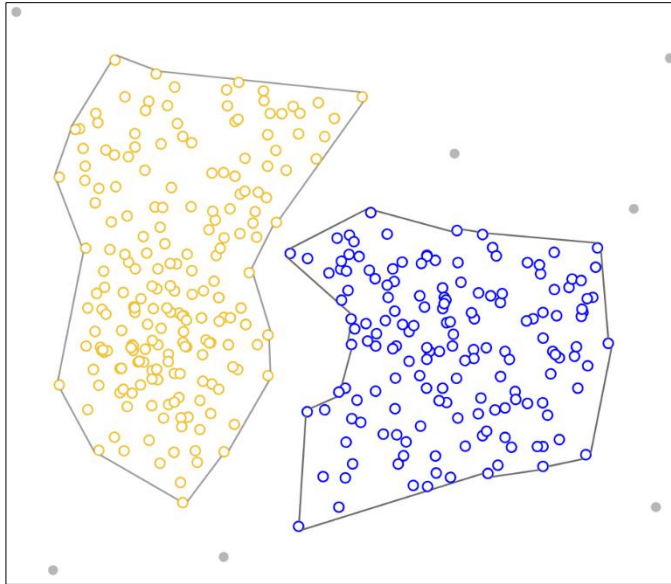
Phương pháp này hoạt động tốt khi:

- các cụm có kích thước tương đương
- dữ liệu không có nhiều ngoại lệ hoặc sự thay đổi mật độ lớn

Tuy nhiên, nó có thể hoạt động kém khi:

- dữ liệu có số chiều cao, các cụm có kích thước hoặc mật độ khác nhau
- tồn tại nhiều outliers

b. Gom cụm dựa trên mật độ (Density-based)



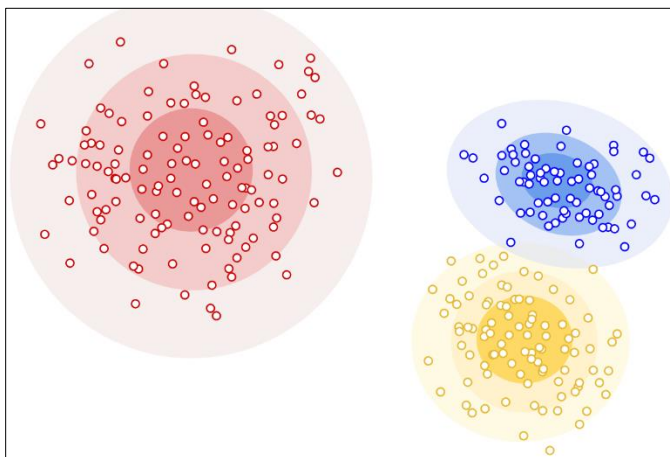
Phương pháp này xác định các cụm bằng cách tìm các vùng có **mật độ dữ liệu cao**, được **ngăn cách bởi các vùng thưa hoặc rỗng**. Khác với các phương pháp khác, nó có thể phát hiện các cụm có **hình dạng bất kỳ**.

Ngoài ra, phương pháp này có khả năng phân biệt giữa các điểm thuộc cụm và các điểm nhiễu (outlier). Nó đặc biệt hữu ích khi dữ liệu có nhiễu hoặc khi không biết trước số lượng cụm.

Trong phương pháp này, các điểm dữ liệu thường được phân thành:

- Điểm lõi (core point)
- Điểm biên (border point)
- Điểm nhiễu (outlier)

c. Gom cụm dựa trên phân phối (Distribution-based)



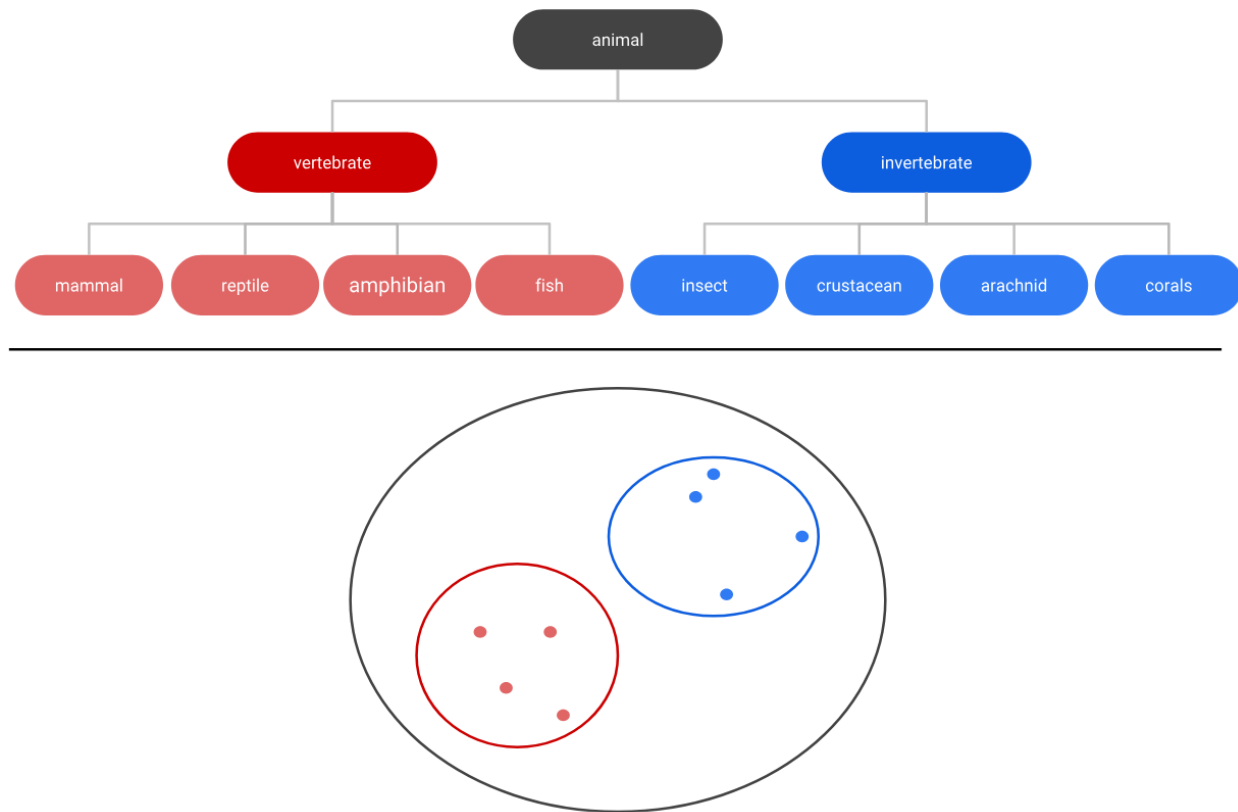
Phương pháp này nhóm các điểm dữ liệu **dựa trên phân phối xác suất** của chúng. Giả định rằng dữ liệu được sinh ra từ các phân phối xác suất (thường là phân phối chuẩn) trong từng chiều.

Khác với phương pháp dựa trên khoảng cách, cách tiếp cận này tìm kiếm các phân phối phù hợp với dữ liệu. Đây là phương pháp dựa trên mô hình, yêu cầu **ước lượng các tham số của phân phối**

nhiều lần, do đó có thể tốn kém tính toán với dữ liệu lớn.

Mỗi điểm dữ liệu được gán một xác suất thuộc về các cụm, cho phép một điểm thuộc nhiều cụm với các mức độ khác nhau.

d. Gom cụm phân cấp (Hierarchical)



Phương pháp này nhóm các điểm dữ liệu dựa trên **mức độ gần nhau** và **mối liên kết giữa chúng**.

- Các điểm càng gần nhau thì càng có xu hướng thuộc cùng một cụm.
- Không cần xác định trước số lượng cụm; thay vào đó, thuật toán xây dựng một cấu trúc dạng cây phân cấp. Cấu trúc này có thể được biểu diễn bằng dendrogram để trực quan hóa các cụm và mối quan hệ giữa chúng.

3. Đánh giá phân cụm

a. Chỉ số đánh giá nội tại

Đây là các thước đo **chỉ sử dụng thông tin bên trong tập dữ liệu**.

- Hữu ích khi làm việc với dữ liệu không có nhãn.
- Chất lượng đánh giá dựa hoàn toàn vào mối quan hệ giữa các điểm dữ liệu.
- Có thể sử dụng khi không có kiến thức trước hoặc nhãn của dữ liệu.

Một số độ đo phổ biến: Sum of Squared Error (SSE), Between Cluster Sum of Squares (BSS), hệ số Silhouette,...

Sum of Squared Error (SSE)

SSE đo độ phân tán bên trong cụm (within-cluster).

$$SSE = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - u_k\|^2$$

Trong đó:

- K : số lượng cụm
- C_k : tập hợp các điểm dữ liệu thuộc cụm k
- x_i : một điểm dữ liệu
- μ_k : tâm của cụm k

Ý nghĩa:

- Tính tổng bình phương khoảng cách từ mỗi điểm dữ liệu đến tâm cụm của nó
- Phản ánh mức độ “chặt chẽ” của các cụm

Diễn giải:

- SSE càng nhỏ $\rightarrow$ các điểm càng gần tâm cụm $\rightarrow$ cụm càng tốt
- SSE thường giảm khi tăng số cụm K

Between Cluster Sum of Squares (BSS)

BSS đo độ tách biệt giữa các cụm (between-cluster).

$$BSS = \sum_{k=1}^K |C_k| \|\mu_k - \mu\|^2$$

Trong đó, μ là tâm của toàn bộ dữ liệu.

Ý nghĩa:

- Đo khoảng cách giữa tâm từng cụm và tâm toàn bộ dữ liệu
- Phản ánh mức độ “xa nhau” giữa các cụm
- BSS càng lớn $\rightarrow$ các cụm càng tách biệt rõ ràng $\rightarrow$ phân cụm tốt

Hệ số Silhouette

Đo mức độ tương đồng và khác biệt của mỗi điểm dữ liệu so với cụm của nó và các cụm khác. Giá trị nằm trong khoảng từ -1 đến +1. Giá trị cao cho thấy điểm dữ liệu phù hợp tốt với cụm của nó và ít phù hợp với các cụm lân cận.



Silhouette đo đồng thời độ chặt trong cụm và độ tách biệt giữa các cụm:

$$S(i) = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Trong đó:

- a_i : khoảng cách trung bình từ điểm i đến các điểm trong cùng cụm (độ chặt)
- b_i : khoảng cách trung bình từ điểm i đến cụm gần nhất khác (độ tách biệt)

Diễn giải:

- Nếu $a_i \ll b_i$: điểm dữ liệu gần với cụm của nó hơn nhiều so với các cụm khác $\rightarrow$ gom cụm tốt
- Nếu $a_i \approx b_i$: điểm nằm giữa các cụm $\rightarrow$ không chắc chắn
- Nếu $a_i > b_i$: điểm có thể bị gán sai cụm

b. Chỉ số đánh giá ngoại tại

Các chỉ số này sử dụng thông tin bên ngoài hoặc nhãn thật (ground truth) để đánh giá hiệu quả của thuật toán. Điều này yêu cầu có dữ liệu nhãn xác định cụm thực sự của mỗi điểm dữ liệu. Khi đó, có thể đánh giá độ chính xác của clustering tương tự như trong bài toán phân lớp.

Một số độ đo phổ biến:

- F-score (F-measure): Đánh giá độ chính xác dựa trên precision và recall khi so sánh kết quả gom cụm với nhãn thật. Giá trị càng cao càng tốt.
- Purity: Đo tỷ lệ các điểm dữ liệu được gán đúng vào cụm tương ứng với lớp thật của chúng. Giá trị càng cao càng tốt.

III. Thuật toán K-Means

1. Tóm tắt lý thuyết

K-Means thực hiện việc phân chia các đối tượng thành các cụm sao cho các đối tượng trong cùng một cụm có sự tương đồng với nhau và khác biệt với các đối tượng thuộc cụm khác.

Thuật toán K-Means chia một tập gồm N mẫu X thành K cụm rời nhau C , trong đó mỗi cụm được mô tả bởi giá trị trung bình μ_j của các mẫu trong cụm. Các giá trị trung bình này thường được gọi là “tâm” của cụm; lưu ý rằng chúng, nói chung, không phải là các điểm thuộc X , mặc dù chúng nằm trong cùng không gian.

Thuật ngữ “K” là một con số. Bạn cần chỉ định cho hệ thống biết bạn muốn tạo ra bao nhiêu cụm. Ví dụ, $K = 2$ tương ứng với hai cụm. Có những cách để xác định giá trị K tốt nhất hoặc tối ưu cho một tập dữ liệu nhất định.

Về mặt bản chất, thuật toán K-Means hướng tới tối ưu hóa hàm mục tiêu:

$$SSE = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - u_k\|^2$$

Các bước hoạt động của thuật toán K-Means:

- **Bước 1. Khởi tạo**
 - Chọn số cụm K
 - Khởi tạo K centroid ban đầu (có thể chọn ngẫu nhiên hoặc cho trước)
- **Bước 2. Gán cụm:** Với mỗi điểm dữ liệu
 - Tính khoảng cách từ điểm đó đến tất cả các centroid
 - Gán điểm vào cụm có centroid gần nhất
- **Bước 3. Cập nhật centroid:** Với mỗi cụm, tính lại centroid bằng cách lấy trung bình tất cả các điểm trong cụm đó:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

- **Bước 4. Lặp lại:** Lặp lại bước 2 và 3 cho đến khi
 - Centroid không còn thay đổi (hội tụ), hoặc
 - Nhãn cụm của các điểm không còn thay đổi
- Kết thúc thuật toán, ta thu được các cụm cuối cùng, vị trí tâm tương ứng và phân loại các điểm dữ liệu vào mỗi cụm.

Công thức tính khoảng cách trong thuật toán K-Means:

Trong thuật toán K-means, khoảng cách được sử dụng phổ biến nhất là **khoảng cách Euclid**. Với 2 điểm trong không gian d chiều:

- Điểm dữ liệu $x = (x_1, x_2, \dots, x_d)$
- Tâm $\mu = (\mu_1, \mu_2, \dots, \mu_d)$

Khoảng cách Euclid giữa chúng được tính bằng công thức:

$$d(x, \mu) = \sqrt{\sum_{i=1}^d (x_i - \mu_i)^2}$$

Trong trường hợp đơn giản nhất là dữ liệu nằm trong không gian 2 chiều: với điểm $x = (x_1, x_2)$ và tâm $\mu = (\mu_1, \mu_2)$, khoảng cách Euclid giữa x và μ là:

$$d(x, \mu) = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2}$$

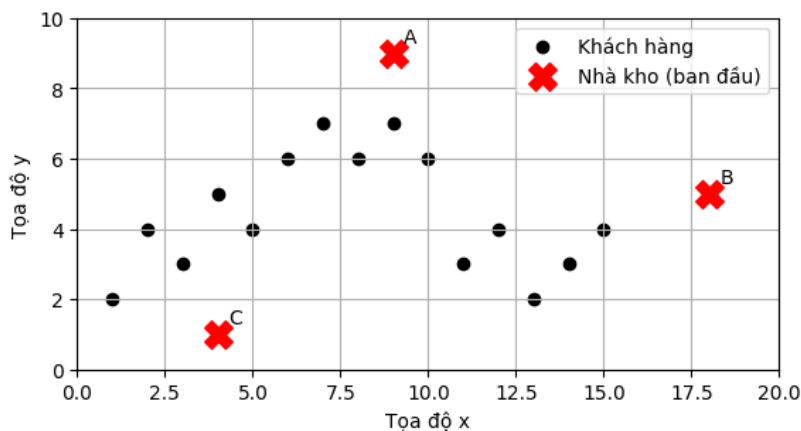
Lưu ý: Trong thực tế, K-means thường không cần tính căn bậc hai vì khi so sánh khoảng cách, ta chỉ cần so sánh bình phương khoảng cách (giảm chi phí tính toán).

Một số loại khoảng cách khác: Manhattan (L1 distance), khoảng cách cosine, ...

2. Ví dụ minh họa

Một công ty logistics đang quản lý danh sách khách hàng trong một thành phố. Mỗi khách hàng được biểu diễn bằng tọa độ hai chiều (x, y) tương ứng với vị trí địa lý của họ:

(1,2), (2,4), (3,3), (4,5), (5,4), (6,6), (7,7), (8,6), (9,7), (10,6), (11,3), (12,4), (13,2), (14,3), (15,4)



Công ty muốn xây dựng **3 trung tâm phân phối ($k = 3$)** nhằm tối ưu hóa việc giao hàng, sao cho mỗi khách hàng được phục vụ bởi trung tâm gần nhất.

Ban đầu, vị trí của 3 trung tâm được giả định tại:

- Trung tâm A: (9, 9)
- Trung tâm B: (18, 5)
- Trung tâm C: (4, 1)

Hãy sử dụng thuật toán **k-means** để:

- Phân cụm các khách hàng thành 3 nhóm dựa trên khoảng cách địa lý
- Cập nhật lại vị trí các trung tâm phân phối (centroid)
- Lặp lại quá trình cho đến khi các cụm không còn thay đổi

Yêu cầu: Xác định các **cụm khách hàng cuối cùng** và **tọa độ các trung tâm phân phối tương ứng** sau khi thuật toán hội tụ.

Ứng dụng K-Means

Vòng lặp 1

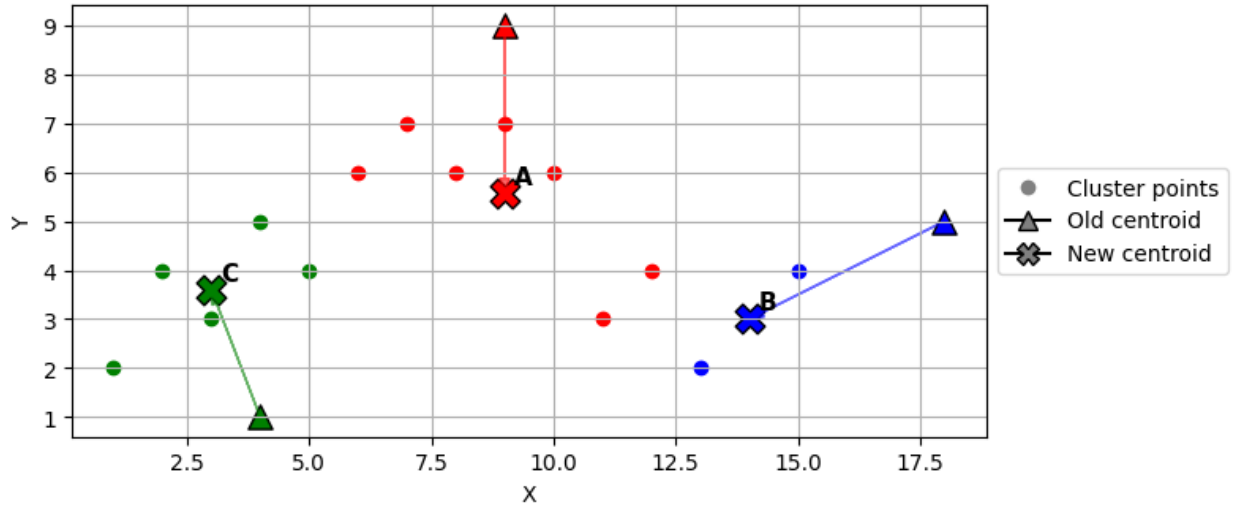
Bước 1: Tính khoảng cách từ mỗi khách hàng đến 3 trung tâm và gán vào trung tâm gần nhất.

Khách hàng	$d(x, A)$	$d(x, B)$	$d(x, C)$	Trung tâm gần nhất
(1, 2)	10.63	17.26	3.16	C
(2, 4)	8.60	16.03	3.61	C
(3, 3)	8.49	15.13	2.24	C
(4, 5)	6.40	14.00	4.00	C
(5, 4)	6.40	13.04	3.16	C
(6, 6)	4.24	12.04	5.39	A

(7, 7)	2.83	11.18	6.71	A
(8, 6)	3.16	10.05	6.40	A
(9, 7)	2.00	9.22	7.81	A
(10, 6)	3.16	8.06	7.81	A
(11, 3)	6.32	7.28	7.28	A
(12, 4)	5.83	6.08	8.54	A
(13, 2)	8.06	5.83	9.06	B
(14, 3)	7.81	4.47	10.20	B
(15, 4)	7.81	3.16	11.40	B

Bước 2: Tính lại vị trí đặt 3 trung tâm (cập nhật tâm cụm)

- Trung tâm A: (6,6), (7,7), (8,6), (9,7), (10,6), (11,3), (12,4)
 - Vị trí cũ: (9, 9)
 - Vị trí mới: (9, 5.57)
 - $x_A = (6 + 7 + 8 + 9 + 10 + 11 + 12)/7 = 9$
 - $y_A = (6 + 7 + 6 + 7 + 6 + 3 + 4)/7 \approx 5.57$
- Trung tâm B: (13,2), (14,3), (15,4)
 - Vị trí cũ (18, 5)
 - Vị trí mới: (14, 3)
 - $x_B = (13 + 14 + 15)/3 = 14$
 - $y_B = (2 + 3 + 4)/3 = 3$
- Trung tâm C: (1,2), (2,4), (3,3), (4,5), (5,4)
 - Vị trí cũ: (4, 1)
 - Vị trí mới: (3, 3.6)
 - $x_C = (1 + 2 + 3 + 4 + 5)/5 = 3$
 - $y_C = (2 + 4 + 3 + 5 + 4)/5 = 3.6$



Vòng lặp 2

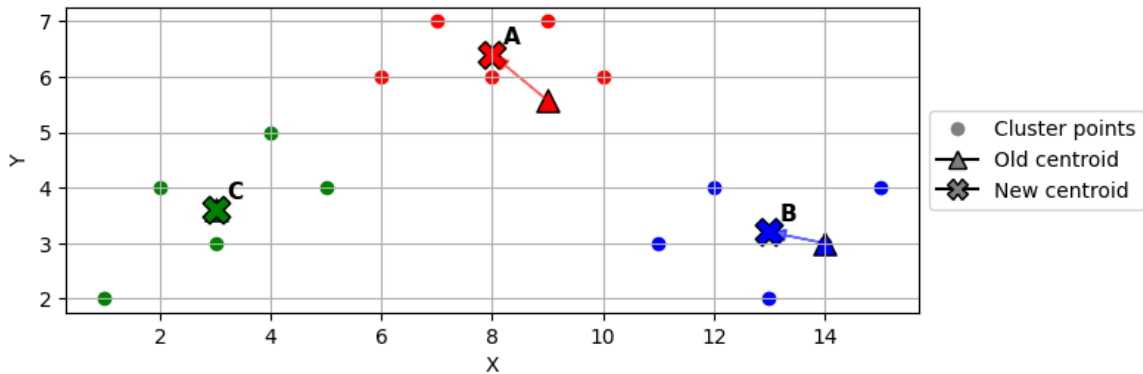
Bước 1: Tính lại khoảng cách giữa mỗi khách hàng đến vị trí đặt trung tâm mới và gán lại trung tâm gần nhất

Khách hàng	$d(x, A)$	$d(x, B)$	$d(x, C)$	Trung tâm gần nhất
(1, 2)	8.76	13.04	2.56	C
(2, 4)	7.17	12.04	1.08	C
(3, 3)	6.53	11.00	0.60	C
(4, 5)	5.03	10.19	1.72	C
(5, 4)	4.30	9.06	2.04	C
(6, 6)	3.03	8.54	3.84	A
(7, 7)	2.46	8.06	5.25	A
(8, 6)	1.09	6.71	5.55	A
(9, 7)	1.43	6.40	6.90	A
(10, 6)	1.09	5.00	7.40	A
(11, 3)	3.26	3.00	8.02	B
(12, 4)	3.39	2.24	9.01	B
(13, 2)	5.36	1.41	10.13	B
(14, 3)	5.62	0.00	11.02	B
(15, 4)	6.20	1.41	12.01	B

Bước 2: Tính lại vị trí đặt 3 trung tâm (cập nhật tâm cụm)

- Trung tâm A: (6,6), (7,7), (8,6), (9,7), (10,6)
 - Vị trí cũ: (9, 5.57)
 - Vị trí mới: (8, 6.4)
 - $x_A = (6 + 7 + 8 + 9 + 10)/5 = 8$

- $y_A = (6 + 7 + 6 + 7 + 6)/5 = 6.4$
- Trung tâm B: (11,3), (12,4), (13,2), (14,3), (15,4)
 - Vị trí cũ (14, 3)
 - Vị trí mới: (13, 3.2)
 - $x_B = (11 + 12 + 13 + 14 + 15)/5 = 13$
 - $y_B = (3 + 4 + 2 + 3 + 4)/5 = 3.2$
- Trung tâm C: (1,2), (2,4), (3,3), (4,5), (5,4)
 - Vị trí cũ: (3, 3.6)
 - Vị trí mới: (3, 3.6)



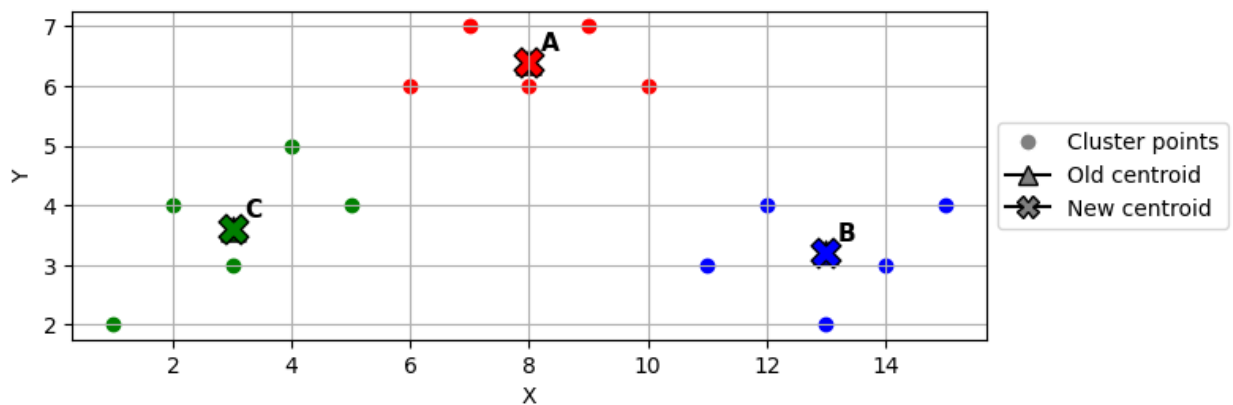
Vòng lặp 3

Bước 1: Tính lại khoảng cách giữa mỗi khách hàng đến vị trí đặt trung tâm mới và gán lại trung tâm gần nhất

Khách hàng	$d(x, A)$	$d(x, B)$	$d(x, C)$	Trung tâm gần nhất
(1, 2)	8.27	12.06	2.56	C
(2, 4)	6.46	11.03	1.08	C
(3, 3)	6.05	10.00	0.60	C
(4, 5)	4.24	9.18	1.72	C
(5, 4)	3.84	8.04	2.04	C
(6, 6)	2.04	7.54	3.84	A
(7, 7)	1.17	7.10	5.25	A
(8, 6)	0.40	5.73	5.55	A
(9, 7)	1.17	5.52	6.90	A
(10, 6)	2.04	4.10	7.40	A
(11, 3)	4.53	2.01	8.02	B
(12, 4)	4.66	1.28	9.01	B
(13, 2)	6.66	1.20	10.13	B
(14, 3)	6.90	1.02	11.02	B
(15, 4)	7.40	2.15	12.01	B

Bước 2: Tính lại vị trí đặt 3 trung tâm (cập nhật tâm cụm)

- Trung tâm A: (6,6), (7,7), (8,6), (9,7), (10,6)
 - Vị trí cũ: (8, 6.4)
 - Vị trí mới: (8, 6.4)
- Trung tâm B: (11,3), (12,4), (13,2), (14,3), (15,4)
 - Vị trí cũ (13, 3.2)
 - Vị trí mới: (13, 3.2)
- Trung tâm C: (1,2), (2,4), (3,3), (4,5), (5,4)
 - Vị trí cũ: (3, 3.6)
 - Vị trí mới: (3, 3.6)



Ta quan sát ở vòng lặp này, không có sự thay đổi phân cụm → **Dừng thuật toán**

Như vậy, kết quả cuối cùng của K-Means:

- Cụm 1
 - Khách hàng: (6,6), (7,7), (8,6), (9,7), (10,6)
 - Trung tâm phân phối A với vị trí (8, 6.4)
- Cụm 2
 - Khách hàng: (11,3), (12,4), (13,2), (14,3), (15,4)
 - Trung tâm phân phối B với vị trí (13, 3.2)
- Cụm 3
 - Khách hàng: (1,2), (2,4), (3,3), (4,5), (5,4)
 - Trung tâm phân phối C với vị trí (3, 3.6)

3. Phương pháp khuỷu tay (ELBOW)

ELBOW là một phương pháp để xác định số cụm cần thiết (tham số k) trong thuật toán K-Means. Đây là một phương pháp heuristic, xem xét k trong một phạm vi nhất định, lần lượt gom cụm theo từng k và tính toán giá trị của tổng bình phương khoảng cách từ các đối

trọng thuộc cụm đến tâm cụm (SSE), hay còn gọi là Within-Cluster Sum of Square (WSS) hoặc Inertia.

Xét tập dữ liệu khách hàng đã cho trong ví dụ minh họa ở phần trước, ta áp dụng K-Means với các giá trị k từ 1 đến 7. Với mỗi giá trị k, thực hiện gom cụm và tính toán SSE.

Ví dụ, với k=3, dựa trên kết quả phân cụm cuối cùng trên ví dụ minh họa, ta tính giá trị SSE như sau:

- **Cụm 1**

- Khách hàng: (6,6), (7,7), (8,6), (9,7), (10,6)
- Trung tâm phân phối A với vị trí (8, 6.4)

$$\begin{aligned}SSE_1 &= (6 - 8)^2 + (6 - 6.4)^2 \\&\quad + (7 - 8)^2 + (7 - 6.4)^2 \\&\quad + (8 - 8)^2 + (6 - 6.4)^2 \\&\quad + (9 - 8)^2 + (7 - 6.4)^2 \\&\quad + (10 - 8)^2 + (8 - 6.4)^2 \approx 11.2\end{aligned}$$

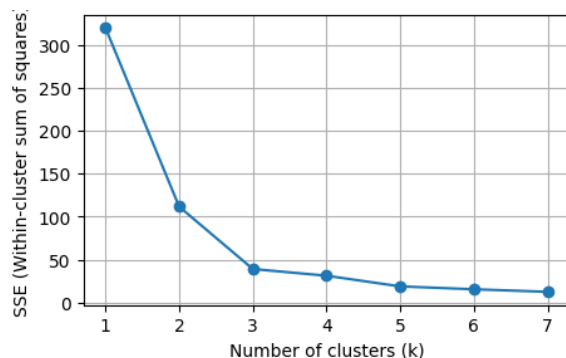
- **Cụm 2**

- Khách hàng: (11,3), (12,4), (13,2), (14,3), (15,4)
- Trung tâm phân phối B với vị trí (13, 3.2)
- $SSE_2 \approx 12.8$

- **Cụm 3**

- Khách hàng: (1,2), (2,4), (3,3), (4,5), (5,4)
- Trung tâm phân phối C với vị trí (3, 3.6)
- $SSE_3 \approx 15.2$

- **Tổng:** $SSE = SSE_1 + SSE_2 + SSE_3 = 39.2$



Kết quả được biểu diễn trên đồ thị với:

- Trục hoành: số cụm k
- Trục tung: giá trị SSE

Quan sát đồ thị, ta thấy SSE giảm mạnh khi tăng k từ 1 lên 3, nhưng sau đó mức giảm trở nên chậm hơn. Điểm “khuyết tay” xuất hiện tại k=3, cho thấy việc tăng thêm số cụm sau giá trị

này không mang lại nhiều cải thiện đáng kể.

Do đó, lựa chọn k=3 là phù hợp cho tập dữ liệu này.

VI. Một số thuật toán gom cụm phổ biến khác

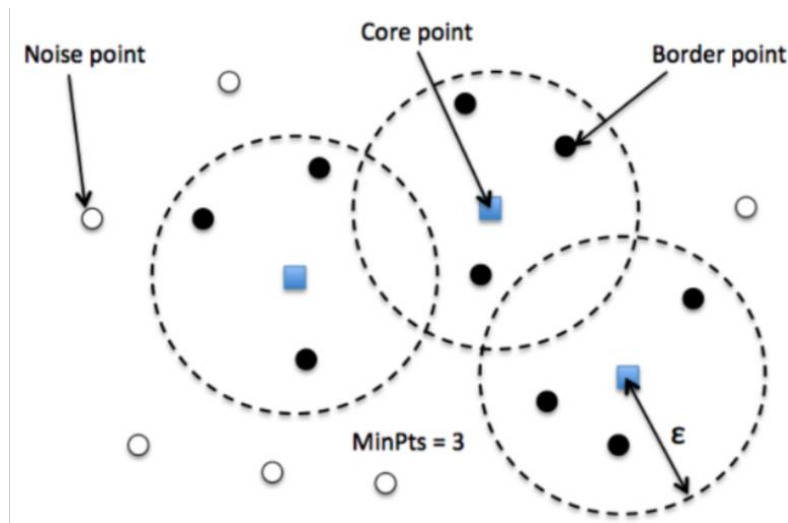
1. Thuật toán DBSCAN

DBSCAN là một thuật toán phân cụm dựa trên mật độ (Density-based). DBSCAN xác định cụm bằng cách mở rộng từ các điểm “đậm đặc”:

- Một điểm được gọi là core point nếu trong một vùng lân cận quanh nó có đủ nhiều điểm khác.
- Từ các core point, thuật toán sẽ mở rộng cụm bằng cách kết nối các điểm lân cận với nhau.
- Những điểm không đủ điều kiện để thuộc cụm nào sẽ bị gán là noise.

Nhờ cách tiếp cận này, DBSCAN có thể:

- Phát hiện các cụm có hình dạng bất kỳ
- Không cần biết trước số cụm k
- Tự động phát hiện outliers



Căn cứ vào vị trí của các điểm dữ liệu so với cụm chúng ta có thể chia chúng thành ba loại:

- Điểm lõi (core point): các điểm nằm sâu bên trong cụm
- Điểm biên (border point): các điểm nằm ở phần ngoài cùng của cụm
- Điểm nhiễu (noise point): điểm không thuộc bất kì một cụm nào

Khi áp dụng DBSCAN, cần chú ý hai tham số chính:

- epsilon (ϵ): bán kính vùng lân cận → Quy định “xa bao nhiêu thì còn được coi là gần”

- minPts: số điểm tối thiểu trong vùng ϵ để tạo thành một cụm $\rightarrow$ Quy định “bao nhiêu điểm thì được coi là đủ đậm đặc”

Các bước của thuật toán DBSCAN khá đơn giản. Thuật toán sẽ thực hiện lan truyền để mở rộng dần phạm vi của cụm cho tới khi chạm tới những điểm biên thì thuật toán sẽ chuyển sang một cụm mới và lặp lại tiếp quá trình trên.

Qui trình của thuật toán:

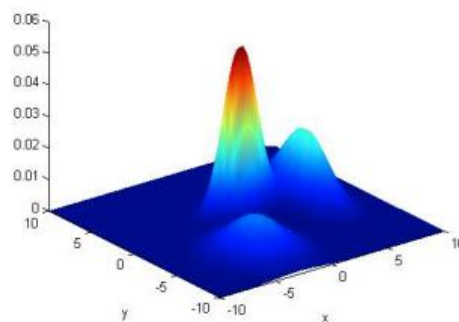
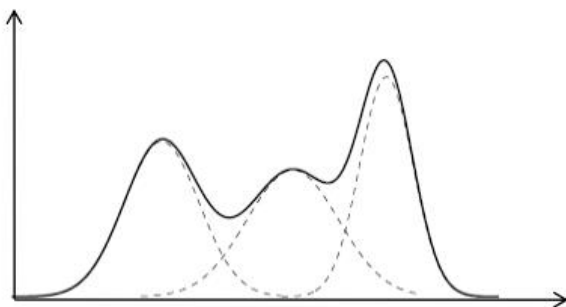
- Bước 1: Thuật toán lựa chọn một điểm dữ liệu bất kì. Sau đó tiến hành xác định các điểm lõi và điểm biên thông qua vùng lân cận epsilon bằng cách lan truyền theo liên kết chuỗi các điểm thuộc cùng một cụm.
- Bước 2: Cụm hoàn toàn được xác định khi không thể mở rộng được thêm. Khi đó lặp lại để qui toàn bộ quá trình với điểm khởi tạo trong số các điểm dữ liệu còn lại để xác định một cụm mới.

Việc chọn tham số trong DBSCAN rất quan trọng vì ảnh hưởng trực tiếp đến kết quả phân cụm.

- minPts: Thường chọn giá trị $\geq$ số chiều dữ liệu + 1, và thực tế hay dùng khoảng 4 trở lên. Nếu dữ liệu có nhiều hoặc lớn, nên chọn giá trị lớn hơn.
- epsilon (ϵ): Có thể chọn bằng cách vẽ biểu đồ k-distance và tìm điểm “khủy tay” (elbow).
 - ϵ quá nhỏ $\rightarrow$ nhiều điểm bị coi là nhiễu
 - ϵ quá lớn $\rightarrow$ các cụm bị gộp lại
- Ngoài ra, việc chọn hàm khoảng cách (thường là Euclidean) cũng ảnh hưởng lớn đến kết quả, nên cần phù hợp với đặc điểm dữ liệu.

2. Mô hình Gaussian Mixture Model

Gaussian Mixture Model (GMM) là một thuật toán gom cụm dựa trên **mô hình xác suất**, trong đó dữ liệu được giả định sinh ra từ nhiều **phân phối Gaussian (chuẩn) đa chiều**, mỗi phân phối tương ứng với một cụm. GMM được gọi là “mixture” vì mỗi điểm dữ liệu không chỉ thuộc về một cụm duy nhất, mà xác suất của nó được tạo thành từ **sự kết hợp của nhiều phân phối Gaussian khác nhau**, giúp mô hình biểu diễn dữ liệu linh hoạt hơn.



Hình trên minh họa phân phối Gaussian đa chiều với số cụm $k=3$ đối với bộ dữ liệu một chiều (bên trái) và hai chiều (bên phải).

Thay vì “cứng” như K-Means (mỗi điểm chỉ thuộc đúng một cụm), GMM sử dụng cách tiếp cận mềm (soft clustering):

- Mỗi điểm dữ liệu không thuộc hoàn toàn vào một cụm, mà có xác suất thuộc về nhiều cụm khác nhau
- Mỗi cụm được mô tả bởi:
 - Trung bình (mean)
 - Độ phân tán (covariance)
 - Trọng số (tỷ lệ xuất hiện của cụm)

Thuật toán sẽ tìm các tham số này sao cho dữ liệu “phù hợp nhất” với mô hình xác suất.

Một ví dụ đơn giản để hình dung về mô hình GMM

Giả sử bạn có dữ liệu về chiều cao và cân nặng của một nhóm người, nhưng không biết trước họ thuộc nhóm nào. Quan sát dữ liệu, bạn thấy có vẻ tồn tại 2 nhóm:

- Nhóm 1: người thấp, nhẹ cân
- Nhóm 2: người cao, nặng cân

Thay vì gán cứng một người vào một nhóm như K-Means, GMM sẽ làm như sau: Với một người có chiều cao 170 cm, cân nặng 65 kg:

- Xác suất thuộc nhóm 1: 30%
- Xác suất thuộc nhóm 2: 70%

Điều này có nghĩa là người này không hoàn toàn thuộc một nhóm duy nhất, mà nằm “ở giữa” hai nhóm

Thuật toán Expectation-Maximization (EM)

GMM thường được huấn luyện bằng thuật toán Expectation-Maximization (EM):

- Bước E (Expectation): Tính xác suất mỗi điểm thuộc về từng cụm
- Bước M (Maximization): Cập nhật lại các tham số của cụm (mean, covariance) dựa trên các xác suất này

Hai bước này được lặp lại cho đến khi hội tụ.

Một số tham số của mô hình GMM

- `n_components`: Số cụm cần chia (giống như `k` trong K-Means).
- `covariance_type`: Cách mô hình mô tả “hình dạng” của mỗi cụm. Hình dạng càng đơn giản, mô hình chạy càng nhanh nhưng kém linh hoạt
 - `full`: mỗi cụm có hình dạng tự do (linh hoạt nhất)
 - `tied`: các cụm dùng chung một hình dạng
 - `diag`: cụm có dạng elip nhưng không xoay (đơn giản hơn)
 - `spherical`: cụm là hình tròn (đơn giản nhất)
- `tol`: Ngưỡng dừng. Nếu mô hình cải thiện quá ít → dừng lại.
- `max_iter`: Số vòng lặp tối đa để tránh chạy quá lâu.
- `init_params`: Cách khởi tạo ban đầu. Mặc định dùng K-Means → giúp bắt đầu “gần đúng” và cải thiện dần bằng GMM.

Cách lựa chọn tham số cho GMM

Để chọn số cụm (và các tham số khác) phù hợp cho GMM, người ta dùng chỉ số BIC (Bayesian Information Criterion). Ý tưởng chính của BIC là:

- Đánh giá mô hình có phù hợp với dữ liệu không
- Đồng thời phạt những mô hình quá phức tạp (nhiều tham số)

Cụ thể, trong thực tế ta thực hiện chọn tham số như sau (ví dụ với `n_components`)

- Thử nhiều giá trị `n_components` (số cụm)
- Với mỗi giá trị, tính BIC
- Chọn mô hình có BIC nhỏ nhất

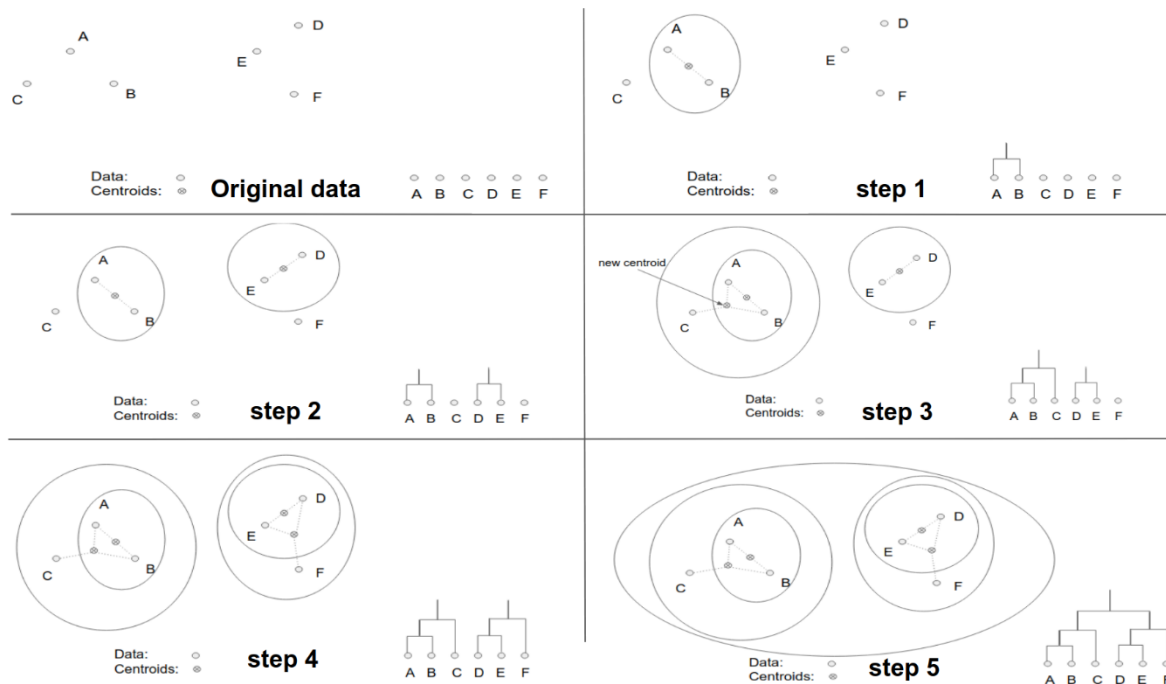
3. Thuật toán Agglomerative

Các chiến lược phân cụm phân cấp chia thành hai mô hình cơ bản: Hợp nhất (agglomerative) và phân chia (divisive).

Chiến lược hợp nhất sẽ đi theo chiều bottom-up (từ dưới lên trên). Quá trình phân cụm bắt đầu ở dưới cùng tại các node lá (còn gọi là leaf node hoặc terminal node). Ban đầu mỗi quan sát sẽ được xem là một cụm tách biệt được thể hiện bởi một node lá. Ở mỗi level chúng ta sẽ tìm cách hợp một cặp cụm thành một cụm duy nhất nhằm tạo ra một cụm mới ở level

cao hơn tiếp theo. Cụm mới này tương ứng với các node quyết định (non-leaf node). Như vậy sau khi hợp cụm thì số lượng cụm ít hơn. Một cặp được chọn để hợp nhất sẽ là những cụm trung gian không giao nhau.

Quá trình phân cụm



Hình trên minh họa quá trình phân cụm phân cấp theo chiến lược agglomerative.

- Mỗi điểm dữ liệu là một chấm tròn
- Tâm cụm là chấm có dấu “x”
- Các đường elip bao quanh các điểm thuộc cùng một cụm
- Biểu đồ dendrogram thể hiện quá trình gộp cụm theo từng bước

Với 6 điểm dữ liệu, thuật toán thực hiện 5 bước gộp:

- Bước 1: Gộp 2 điểm gần nhau nhất thành một cụm
- Bước 2: Chọn điểm gần cụm đã có nhất → gộp vào cụm đó
- Bước 3–4: Tiếp tục gộp điểm/cụm gần nhất vào cụm hiện có
- Bước 5: Gộp các cụm lớn lại với nhau → tạo thành cụm cuối cùng chứa toàn bộ dữ liệu

Ý tưởng cốt lõi

- Thuật toán bắt đầu từ các điểm riêng lẻ, sau đó gộp dần các điểm/cụm gần nhau cho đến khi tất cả thành một cụm lớn.

- Khoảng cách giữa các cụm được tính bằng khoảng cách Euclidean giữa tâm cụm, trong đó tâm cụm là trung bình của các điểm trong cụm.

Phương pháp xác định khoảng cách giữa 2 cụm

Ward linkage là cách đo khoảng cách giữa hai cụm dựa trên mức độ “tăng thêm độ phân tán” (phương sai) khi gộp chúng lại. Ý tưởng chính:

- Khi gộp hai cụm, nếu dữ liệu vẫn gọn và ít phân tán → hai cụm này gần nhau
- Nếu gộp lại làm dữ liệu phân tán nhiều hơn → hai cụm này khác nhau

Như vậy, thuật toán sẽ luôn chọn gộp hai cụm làm tăng phương sai ít nhất.

Một số phương pháp xác định khoảng cách khác: single linkage, complete linkage, group average.

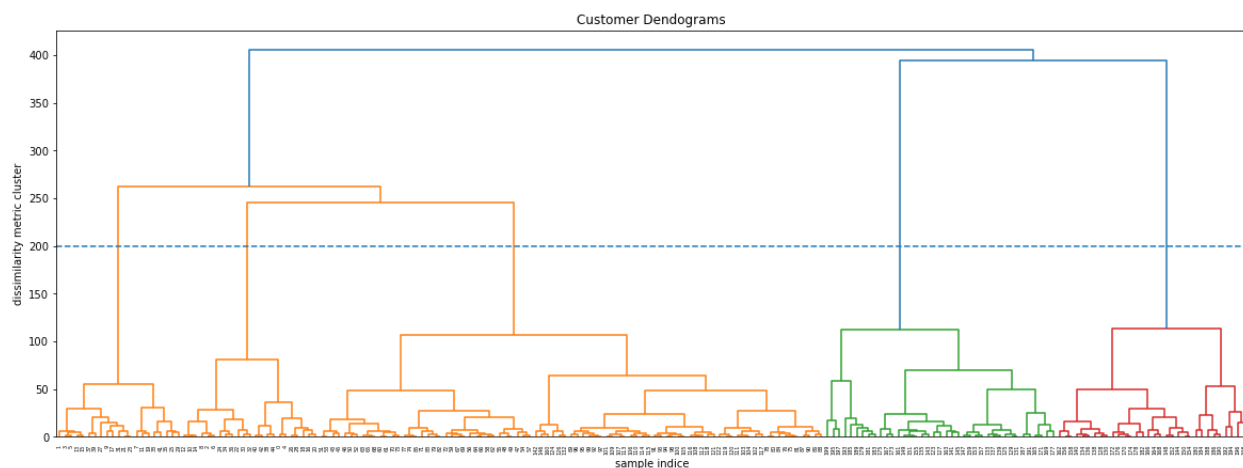
Biểu đồ dendrogram

Dendrogram là biểu đồ dạng cây dùng để trực quan quá trình gộp cụm trong phân cụm phân cấp.

- Trục dọc (chiều cao) thể hiện mức độ khác biệt giữa các cụm
- Hai nhánh càng nối với nhau ở độ cao thấp → hai cụm càng giống nhau
- Nối ở độ cao cao → hai cụm khác nhau nhiều

Để chọn số cụm:

- Kẻ một đường ngang trên biểu đồ
- Đếm số nhánh bị cắt → đó là số cụm phù hợp



Trong biểu đồ dendrogram ở trên, trục hoành biểu diễn thứ tự các điểm dữ liệu, còn trục tung thể hiện mức độ khác biệt giữa các cụm, được tính bằng Ward linkage.

Quan sát dendrogram, ta thấy có một khoảng nhảy lớn (khoảng cách tăng mạnh) ở gần mức 200. Điều đó cho thấy:

- Nếu gộp tiếp (vượt qua 200) → các cụm bắt đầu rất khác nhau
- Nếu dừng tại 200 → các cụm vẫn còn tương đối giống nhau trong nội bộ

Khi kẻ đường ngang tại 200 và đếm số nhánh bị cắt, ta được 5 cụm

Như vậy, ý tưởng cốt lõi khi sử dụng biểu đồ dendrogram để chọn số cụm:

- Chọn mức cắt ở nơi có khoảng cách tăng đột ngột (elbow/step lớn)
- Vì đó là điểm “ranh giới” giữa gộp cụm hợp lý và gộp cụm quá mức (làm mất ý nghĩa)

V. Thực hành

Bài tập 1. Sử dụng Python cài đặt thuật toán K-Means

Sử dụng Python để cài đặt từng bước tính toán của thuật toán K-Means trong ví dụ minh họa ở nội dung lý thuyết (Phần III. Thuật toán K-Means, Mục 2. Ví dụ minh họa)

Câu 1. Khởi tạo dữ liệu và cấu trúc cụm

- Khởi tạo tập dữ liệu vị trí khách hàng X dưới dạng numpy array
- Khởi tạo số cụm $k = 3$
- Khởi tạo cụm ban đầu $\text{init_centers} = \{\text{cluster_id: } \{\dots\}, \dots\}$
- Xây dựng cấu trúc dữ liệu cho mỗi cụm dưới dạng dictionary:

```
{  
    'center': ..., # tọa độ tâm cụm (numpy array)  
    'points': []  # danh sách các điểm thuộc cụm  
}
```

Câu 2. Định nghĩa hàm **distance(p1, p2)** để tính khoảng cách Euclid giữa 2 điểm dữ liệu

Gợi ý:

- *np.sqrt()*: Tính căn bậc hai của một số hoặc của từng phần tử trong một mảng (theo từng phần tử).
- *np.sum()*: Tính tổng tất cả các phần tử trong một mảng hoặc theo một trục (axis) được chỉ định.

Câu 3. Định nghĩa hàm **assign_clusters(X, clusters)** để gán từng điểm dữ liệu đến cụm gần nhất

Hàm nhận vào:

- **X**: mảng dữ liệu gồm N điểm (dạng numpy array, kích thước Nx2)
- **clusters**: danh sách các cụm (với kiểu dữ liệu như đã quy định ở Câu 1)

Giá trị trả về: **clusters** (danh sách các cụm, trong đó đã được cập nhật danh sách *points* cho từng cụm)

Chức năng chính của hàm:

- Tính khoảng cách từ N điểm đến tất cả các tâm cụm bằng hàm *distance(p1, p2)* (đã cài đặt ở Câu 2)
- Lưu các khoảng cách vào một danh sách (*dist = []*)
- Xác định cụm gần nhất bằng *np.argmin(dist)*
- Gán điểm vào cụm tương ứng (thêm vào danh sách *points* của cụm đó)

Các chức năng khác:

- Trước khi bắt đầu gán cụm ở mỗi lần gọi hàm, reset danh sách *points* của tất cả các cụm về rỗng (*clusters[i]['points'] = []*)
- Trong quá trình thực hiện, cần in ra thông tin chi tiết:
 - Với mỗi khách hàng, in ra vị trí và khoảng cách đến từng cụm
 - Cụm mà khách hàng đó được gán vào
 - Ví dụ:

```
Point [1 2]:
Distance to Cluster 0 (center=[...]): ...
Distance to Cluster 1 (center=[...]): ...
Distance to Cluster 2 (center=[...]): ...
→ Assigned to Cluster X
```

Câu 4. Định nghĩa hàm **update_clusters(clusters)** để cập nhật lại tâm mới của từng cụm dựa trên giá trị trung bình của các điểm gán cho mỗi cụm.

Hàm nhận vào: *clusters* (danh sách các cụm)

Hàm trả về: *clusters* (trong đó giá trị *center* đã được cập nhật)

Chức năng chính: Với mỗi cụm,

- Nếu cụm có ít nhất 1 điểm:
 - Tính tâm cụm mới bằng trung bình cộng theo từng chiều
 - *new_center = points.mean(axis=0)*
 - Cập nhật lại giá trị *center* của cụm

- Nếu cụm không có điểm nào thì giữ nguyên tâm cụm (không cập nhật)

Trong quá trình thực hiện, cần in ra thông tin chi tiết cho từng cụm:

- Danh sách các điểm thuộc cụm
- Tâm cụm trước khi cập nhật
- Tâm cụm sau khi cập nhật

Ví dụ:

Cluster 0:

Points: [[...], [...], ...]

Old center: [...]

New center: [...]

Trường hợp cụm không có điểm nào:

Cluster i:

Points: []

No points assigned → center unchanged

Câu 5. Định nghĩa hàm **kmeans(X, init_centers, max_iters, tol)** để lặp toàn bộ thuật toán K-means cho đến khi hội tụ

Hàm nhận vào:

- *X*: tập dữ liệu đầu vào dạng numpy array kích thước (N, 2)
- *init_centers*: các tâm cụm ban đầu (dạng dictionary)
- *max_iters*: số vòng lặp tối đa của thuật toán
- *tol*: ngưỡng hội tụ (sai số cho phép)

Hàm trả về: *clusters*, trong đó:

- *center*: là tâm cụm cuối cùng
- *points*: là các điểm thuộc từng cụm sau khi hội tụ

Chức năng của hàm

- Khởi tạo cụm
 - Từ *init_centers*, tạo danh sách clusters
 - Số cụm k được xác định bằng $k = \text{len}(\text{init_centers})$

- In ra các tâm cụm ban đầu, ví dụ:


```
=== Initial Centers ===
Cluster 0: [...]
Cluster 1: [...]
...
```
- Lặp thuật toán: Lặp tối đa *max_iters* lần (*for iteration in range(max_iters)*), Trong mỗi vòng lặp, thực hiện các bước
 - Gán cụm (gọi hàm *assign_clusters* đã cài đặt ở Câu 3)
 - Lưu lại tâm cụm cũ trước khi cập nhật


```
prev_centers = [c['center'].copy() for c in clusters]
```
 - Cập nhật tâm cụm (gọi hàm *update_clusters* đã cài đặt ở Câu 4)
 - Kiểm tra hội tụ: Với mỗi cụm, tính độ dịch chuyển của tâm cụm


```
shift = np.linalg.norm(new_center - prev_center)
```

 Nếu tất cả các độ dịch chuyển nhỏ hơn *tol* thì kết luận thuật toán đã hội tụ và dừng vòng lặp

Bài tập 2. Cài đặt thuật toán K-Means bằng thư viện sklearn

Cho dữ liệu mua sắm của khách hàng ([Mall Customer Segmentation Data](#)) bao gồm các thông tin sau: Giới tính (Gender), Tuổi (Age), Thu nhập hằng năm (Annual income – đơn vị tính: nghìn đô) và Số điểm mua sắm (Spending Score – từ 1 đến 100). Sinh viên hãy thực hiện những yêu cầu sau đây

Câu 1. Nhập dữ liệu, thống kê các thông tin cơ bản, xử lý dữ liệu bị thiếu.

Câu 2. Sử dụng biểu đồ histogram để biểu diễn phân phối của lần lượt các thuộc tính Tuổi, Thu nhập hằng năm và Số điểm mua sắm

Câu 3. Thống kê số mẫu dữ liệu theo thuộc tính Giới tính.

Câu 4. Sử dụng biểu đồ phân tán (scatter plot) để khảo sát phân bố của thuộc tính Tuổi và Thu nhập hằng năm theo Giới tính.

Câu 5. Tương tự Câu 4, khảo sát thuộc tính Thu nhập hằng năm và Số điểm mua sắm theo Giới tính

Câu 6. Chọn thuộc tính Tuổi và Số điểm mua sắm để gom cụm. Sử dụng phương pháp khuỷu tay (ELBOW method) để xác định số cụm cần thiết. Tiến hành gom cụm bằng thuật toán K-Means với số cụm vừa xác định

Gợi ý: Sử dụng *sklearn.KMeans.inertia_*

Câu 7. Biểu diễn kết quả gom cụm trên bằng biểu đồ

Câu 8. Đánh giá kết quả gom cụm bằng Hệ số Silhouette

Gợi ý: Sử dụng hàm có sẵn trong thư viện sklearn

```
Sklearn.metrics.silhouette_score(X, labels, metric="euclidean")
```

Câu 9. Thực hiện tương tự câu 6-8 với trường hợp gom cụm theo các thuộc tính:

- Thu nhập hằng năm và Số điểm mua sắm
- Tuổi, Thu nhập hằng năm và Số điểm mua sắm (thực hiện vẽ biểu đồ 3D để biểu diễn kết quả gom cụm)

Gợi ý: Sử dụng thư viện plotly để vẽ biểu đồ 3D

Bài tập 3. Thực hành trên nhiều thuật toán gom cụm

Để kiểm tra xem khối u, tổn thương trong ngực bệnh nhân có phải là ung thư hay không, người ta thực hiện phương pháp chọc hút tế bào bằng kim nhỏ (FNA). Tế bào lấy được sau đó được phân tích dưới kính hiển vi. Bảng dữ liệu Breast Cancer được tính từ hình ảnh dưới kính hiển vi, các thuộc tính trong bảng mô tả các đặc tính của tế bào được phân tích. Sinh viên hãy dùng thuật toán gom cụm để gom nhóm các khối u lành tính (benign) hoặc ác tính (malignant)

Câu 1. Đọc dữ liệu Breast Cancer bằng pandas. Hiển thị 5 dòng đầu tiên và kiểm tra kích thước của dataset. Xác định số lượng thuộc tính và số lượng mẫu.

Câu 2. Phân tích và khám phá sơ bộ dữ liệu

- Sử dụng data.info() và data.describe() để:
 - Kiểm tra kiểu dữ liệu
 - Phân tích phân phối của các thuộc tính
 - Nhận xét sơ bộ về outlier và sự chênh lệch scale giữa các feature
- Kết hợp trực quan hóa dữ liệu để hỗ trợ phân tích:
 - Vẽ histogram một số thuộc tính tiêu biểu để quan sát phân phối
 - Vẽ boxplot để phát hiện outlier

Câu 3. Thực hiện tiền xử lý dữ liệu theo các bước:

- Tạo ma trận đặc trưng X
- Phân tích tương quan giữa các features (correlation matrix)
- Nhận xét các nhóm feature có khả năng dư thừa thông tin
- Lựa chọn/loại bỏ feature dựa trên mức tương quan cao để giảm chiều dữ liệu và cải thiện hiệu quả thuật toán gom cụm

Câu 4. Trực quan hóa dữ liệu và chuẩn hóa

- Chuẩn hóa dữ liệu bằng StandardScaler để tạo X_{scaled}
- Trực quan hóa dữ liệu trước và sau chuẩn hóa bằng PCA (2D)
- Nhận xét ảnh hưởng của scaling đến phân bố dữ liệu

Câu 5. Xác định số cụm bằng Elbow Method

- Vẽ đồ thị SSE theo các giá trị k
- Phân tích điểm “elbow”
- Chọn giá trị k phù hợp

Lưu ý: Trong bài toán này, k được kỳ vọng là 2

Câu 6. Đánh giá ảnh hưởng của chuẩn hóa

- Thực hiện K-Means với $k = 2$ trên:
 - Dữ liệu gốc X
 - Dữ liệu chuẩn hóa X_{scaled}
- So sánh bằng:
 - Silhouette Score
 - Trực quan PCA 2D
- Nhận xét vai trò của feature scaling trong clustering.

Câu 7. Huấn luyện K-Means và phân tích độ ổn định

- Huấn luyện K-Means với $k = 2$
- Thử nghiệm thêm $k = 3$ và $k = 4$
- So sánh Silhouette Score và sự thay đổi cluster
- Nhận xét mức độ ổn định của K-Means trên dataset nhiều chiều.

Câu 8. DBSCAN với k -distance graph

- Sử dụng X_{scaled}
- Chọn min_samples ban đầu (5–10)
- Vẽ k -distance graph để chọn eps
- Huấn luyện DBSCAN
- In:
 - số cụm (không tính noise)
 - số điểm noise
 - phân bố điểm trong từng cụm

- Trực quan hóa kết quả bằng PCA

Câu 9. Tinh chỉnh DBSCAN (Grid Search)

- Thử nhiều giá trị `eps` và `min_samples`
- Ghi nhận số cụm và số noise
- Phân tích xu hướng thay đổi theo tham số
- Chọn cấu hình hợp lý nhất
- Trực quan hóa kết quả tốt nhất bằng PCA
- Nhận xét ưu/nhược DBSCAN trên dữ liệu này

Câu 10. Gaussian Mixture Model (GMM)

- Huấn luyện GMM với `n_components = 2`
- Gán nhãn cụm cho dữ liệu
- So sánh với K-Means về cách phân cụm (hard vs soft clustering)

Câu 11. Chọn số cụm bằng BIC

- Huấn luyện GMM với nhiều giá trị `n_components`
- Vẽ biểu đồ BIC
- Chọn mô hình có BIC nhỏ nhất
- Nhận xét số cụm tối ưu của dữ liệu

Câu 12. Trực quan hóa GMM vs K-Means

- Giảm chiều dữ liệu bằng PCA
- Vẽ kết quả GMM và K-Means trên cùng không gian
- So sánh:
 - hình dạng cụm
 - mức độ tách biệt

Câu 13. Agglomerative Clustering

- Huấn luyện với `k = 2`
- Thử các linkage: ward, complete, average
- So sánh kết quả giữa các linkage

Câu 14. Phân tích dendrogram

- Vẽ dendrogram bằng Ward linkage
- Xác định số cụm bằng cách cắt cây

- So sánh số cụm từ dendrogram với K-Means và GMM
- Nhận xét cấu trúc phân cấp của dữ liệu

Câu 15. So sánh toàn diện các thuật toán

- So sánh: K-Means, DBSCAN, GMM, Hierarchical
- Dựa trên:
 - số cụm tạo ra
 - Silhouette Score
 - độ ổn định
 - khả năng phát hiện cấu trúc dữ liệu
- Cuối cùng:
 - so sánh với nhãn thật (diagnosis)
 - đánh giá mức độ phù hợp của từng thuật toán với bài toán y sinh